

Database & ORM Command Reference

[Prisma](#) · [MongoDB](#) · [PostgreSQL](#) · [SQL](#)

[Install](#) · [Generate](#) · [Seed](#) · [Insert](#) · [Update](#) · [Delete](#) · [Select](#)

Contents

1.	Prisma ORM — install, generate, migrate, seed, full CRUD	3
2.	MongoDB — native driver, insert / find / update / delete	8
3.	PostgreSQL — psql & node-postgres, full CRUD	11
4.	SQL — core reference (DDL, DML, joins, aggregates)	14

Note: Page numbers are approximate. Prisma v7 does not yet support MongoDB as of this writing — use Prisma v6.x for MongoDB projects, and v7 (or v6) for PostgreSQL. Details are called out where behavior differs.

1 | Prisma ORM

1.1 Install & Initialize

```
# Install CLI + client
npm install prisma --save-dev
npm install @prisma/client

# Initialize a project (creates prisma/schema.prisma, .env, prisma.config.ts)
npx prisma init --datasource-provider postgresql
npx prisma init --datasource-provider mongodb
```

1.2 Schema Basics (prisma/schema.prisma)

```
generator client {
  provider = "prisma-client-js"      // or "prisma-client" for v7 ESM client
  output   = "../generated/prisma"
}

datasource db {
  provider = "postgresql"             // "mysql" | "sqlite" | "sqlserver" | "mongodb"
  url      = env("DATABASE_URL")
}

model User {
  id          Int      @id @default(autoincrement())
  email       String   @unique
  name        String?
  posts       Post[]
  createdAt   DateTime @default(now())
}

model Post {
  id          Int      @id @default(autoincrement())
  title       String
  authorId    Int
  author      User     @relation(fields: [authorId], references: [id])
}

// MongoDB models use ObjectId ids instead of autoincrement:
model Product {
  id    String @id @default(auto()) @map("_id") @db.ObjectId
  name  String
  price Float
}
```

1.3 prisma generate

```
# Generate/refresh the Prisma Client from the schema
npx prisma generate

# Generate using an explicit config file (Prisma v7)
npx prisma generate --config prisma.config.ts

# Watch mode while editing the schema
npx prisma generate --watch
```

Note: Run `prisma generate` after every `schema.prisma` change and after every fresh `npm install` (it is not run automatically by `npm`).

1.4 Migrations (SQL databases only)

```
# Create + apply a new migration in development, and regenerate the client
npx prisma migrate dev --name init

# Apply pending migrations in production/CI (non-interactive)
npx prisma migrate deploy

# Check migration status
npx prisma migrate status

# Reset the database (drops data), reapplies migrations, then reseeds
npx prisma migrate reset

# Push schema straight to the DB without creating a migration file
# (used for MongoDB, prototyping, or SQLite scratch DBs)
npx prisma db push
```

Note: MongoDB has no schema-migration engine — Prisma Migrate is SQL-only. For MongoDB always use `prisma db push` to sync the schema.

1.5 Seeding the Database

Add a seed script and point Prisma at it, then run it directly or automatically on `migrate reset`.

package.json (Prisma v6 style)

```
{
  "prisma": {
    "seed": "tsx prisma/seed.ts"
  }
}
```

prisma.config.ts (Prisma v7 style)

```
import { defineConfig, env } from "prisma/config";

export default defineConfig({
  schema: "prisma/schema.prisma",
  migrations: {
    path: "prisma/migrations",
    seed: "tsx prisma/seed.ts",
  },
  datasource: { url: env("DATABASE_URL") },
});
```

prisma/seed.ts

```
import { PrismaClient } from "@prisma/client";
const prisma = new PrismaClient();

async function main() {
  await prisma.user.createMany({
    data: [
      { email: "alice@example.com", name: "Alice" },
      { email: "bob@example.com", name: "Bob" },
    ],
    skipDuplicates: true,
  });
}
```

```
    });  
  }  
  
  main()  
    .then(() => prisma.$disconnect())  
    .catch(async (e) => {  
      console.error(e);  
      await prisma.$disconnect();  
      process.exit(1);  
    });  
}
```

Run the seed

```
npx prisma db seed      # run the configured seed script directly  
npx prisma migrate reset # wipes DB, re-applies migrations, then auto-seeds
```

1.6 Prisma Client — Full CRUD Reference

All examples assume: `import { PrismaClient } from "@prisma/client"; const prisma = new PrismaClient();`

Create

```
// Single record
const user = await prisma.user.create({
  data: { email: "jane@example.com", name: "Jane" },
});

// Nested create (with a related record)
const userWithPost = await prisma.user.create({
  data: {
    email: "sam@example.com",
    posts: { create: [{ title: "Hello world" }] },
  },
  include: { posts: true },
});

// Bulk insert
await prisma.user.createMany({
  data: [{ email: "a@x.com" }, { email: "b@x.com" }],
  skipDuplicates: true,
});

// Create and return created rows (Postgres/SQLite/CockroachDB)
const created = await prisma.user.createManyAndReturn({
  data: [{ email: "c@x.com" }, { email: "d@x.com" }],
});
```

Read / Select

```
// Unique lookup
const user = await prisma.user.findUnique({ where: { id: 1 } });

// First match
const first = await prisma.user.findFirst({ where: { name: "Jane" } });

// Many, with filter/sort/pagination/select
const users = await prisma.user.findMany({
  where: { email: { contains: "@example.com" } },
  select: { id: true, email: true },
  orderBy: { createdAt: "desc" },
  skip: 0,
  take: 20,
});

// Include relations
const withPosts = await prisma.user.findMany({ include: { posts: true } });

// Count / aggregate / group
const total = await prisma.user.count({ where: { name: { not: null } } });
const stats = await prisma.post.aggregate({ _count: true, _avg: { views: true } });
const grouped = await prisma.post.groupBy({
  by: ["authorId"],
  _count: { _all: true },
});
```

Update

```
// Update one record
await prisma.user.update({
  where: { id: 1 },
  data: { name: "Jane Doe" },
});

// Update many
await prisma.user.updateMany({
  where: { name: null },
  data: { name: "Unknown" },
});

// Update and return rows (Postgres/SQLite/CockroachDB)
await prisma.user.updateManyAndReturn({
  where: { name: null },
  data: { name: "Unknown" },
});

// Upsert (update if exists, otherwise create)
await prisma.user.upsert({
  where: { email: "jane@example.com" },
  update: { name: "Jane Doe" },
  create: { email: "jane@example.com", name: "Jane Doe" },
});

// Atomic numeric update
await prisma.post.update({
  where: { id: 1 },
  data: { views: { increment: 1 } },
});
```

Delete

```
// Delete one
await prisma.user.delete({ where: { id: 1 } });

// Delete many
await prisma.user.deleteMany({ where: { name: "Unknown" } });

// Delete every row in a table (use with care)
await prisma.user.deleteMany({});
```

Transactions & Raw SQL

```
// Sequential transaction (all-or-nothing)
await prisma.$transaction([
  prisma.user.update({ where: { id: 1 }, data: { name: "A" } }),
  prisma.post.deleteMany({ where: { authorId: 1 } }),
]);

// Interactive transaction
await prisma.$transaction(async (tx) => {
  const u = await tx.user.create({ data: { email: "x@x.com" } });
  await tx.post.create({ data: { title: "hi", authorId: u.id } });
});

// Raw queries (escape-hatch for SQL databases)
const rows = await prisma.$queryRaw`SELECT * FROM "User" WHERE id = ${1}`;
await prisma.$executeRaw`UPDATE "User" SET name = ${'X'} WHERE id = ${1}`;
```

1.7 CLI Quick Reference

Command	Purpose
prisma init	Scaffold schema.prisma, .env, prisma.config.ts
prisma generate	(Re)generate Prisma Client from the schema
prisma db push	Push schema to DB without a migration (MongoDB/prototyping)
prisma db pull	Introspect an existing DB into schema.prisma
prisma db seed	Run the configured seed script
prisma migrate dev	Create + apply a migration in development
prisma migrate deploy	Apply pending migrations in production/CI
prisma migrate reset	Drop, recreate, migrate, then reseed the DB
prisma migrate status	Show applied / pending migrations
prisma studio	Open a GUI to browse and edit data (SQL databases)
prisma format	Format schema.prisma
prisma validate	Validate schema.prisma syntax

Note: Prisma Studio and Prisma Migrate do not currently support MongoDB. For MongoDB, use db push, and explore data with MongoDB Compass, Atlas, or the mongo shell instead of Studio.

2 | MongoDB

2.1 Connect (Node.js driver)

```
npm install mongodb
```

```
import { MongoClient } from "mongodb";

const client = new MongoClient(process.env.MONGO_URL);
await client.connect();
const db = client.db("shop");
const users = db.collection("users");
```

2.2 Insert

```
// Insert one document
await users.insertOne({ email: "jane@example.com", name: "Jane" });

// Insert many documents
await users.insertMany([
  { email: "a@x.com", name: "A" },
  { email: "b@x.com", name: "B" },
]);
```

2.3 Select / Find

```
// Find one
const user = await users.findOne({ email: "jane@example.com" });

// Find many, with filter, projection, sort, and pagination
const cursor = users
  .find({ age: { $gte: 18 } }) // filter
  .project({ email: 1, name: 1, _id: 0 }) // projection
  .sort({ name: 1 }) // sort ascending
  .skip(0)
  .limit(20);
const results = await cursor.toArray();

// Common query operators
// $eq $ne $gt $gte $lt $lte $in $nin $exists $regex $and $or $not
await users.find({ name: { $in: ["Jane", "Bob"] } }).toArray();

// Count documents
const total = await users.countDocuments({ age: { $gte: 18 } });

// Aggregation pipeline
const stats = await users.aggregate([
  { $match: { age: { $gte: 18 } } },
  { $group: { _id: "$country", count: { $sum: 1 } } },
  { $sort: { count: -1 } },
]).toArray();
```

2.4 Update

```
// Update one document
await users.updateOne(
```

```

    { email: "jane@example.com" },
    { $set: { name: "Jane Doe" } }
  );

// Update many documents
await users.updateMany(
  { active: false },
  { $set: { status: "inactive" } }
);

// Common update operators
// $set $unset $inc $mul $push $pull $addToSet $rename $currentDate
await users.updateOne(
  { _id: id },
  { $inc: { loginCount: 1 }, $push: { history: "login" } }
);

// Replace an entire document
await users.replaceOne({ _id: id }, { email: "new@example.com", name: "New" });

// Upsert (update if found, otherwise insert)
await users.updateOne(
  { email: "new@example.com" },
  { $set: { name: "New" } },
  { upsert: true }
);

// findOneAndUpdate (returns the document)
const updated = await users.findOneAndUpdate(
  { email: "jane@example.com" },
  { $set: { name: "Jane D." } },
  { returnDocument: "after" }
);

```

2.5 Delete

```

// Delete one document
await users.deleteOne({ email: "jane@example.com" });

// Delete many documents
await users.deleteMany({ status: "inactive" });

// findOneAndDelete (returns the deleted document)
const deleted = await users.findOneAndDelete({ email: "jane@example.com" });

// Drop the entire collection
await users.drop();

```

2.6 Indexes

```

await users.createIndex({ email: 1 }, { unique: true });
await users.createIndex({ name: 1, age: -1 }); // compound index
await users.dropIndex("email_1");
await users.listIndexes().toArray();

```

2.7 mongosh (shell) Equivalents

```

use shop
db.users.insertOne({ email: "a@x.com" })
db.users.find({ age: { $gte: 18 } }).sort({ name: 1 }).limit(10)

```

```
db.users.updateOne({ email: "a@x.com" }, { $set: { name: "A" } })
db.users.deleteOne({ email: "a@x.com" })
db.users.countDocuments()
show collections
```

3 | PostgreSQL

3.1 Connect

psql (command line)

```
psql -h localhost -U postgres -d mydb
```

node-postgres (Node.js)

```
npm install pg
```

```
import { Pool } from "pg";
const pool = new Pool({ connectionString: process.env.DATABASE_URL });
```

3.2 Insert

```
-- Single row
INSERT INTO users (email, name) VALUES ('jane@example.com', 'Jane');

-- Multiple rows
INSERT INTO users (email, name) VALUES
  ('a@x.com', 'A'),
  ('b@x.com', 'B');

-- Insert and return the new row
INSERT INTO users (email, name) VALUES ('c@x.com', 'C') RETURNING *;

// node-postgres
await pool.query(
  "INSERT INTO users (email, name) VALUES ($1, $2) RETURNING **",
  ["jane@example.com", "Jane"]
);
```

3.3 Select

```
SELECT * FROM users WHERE age >= 18 ORDER BY name ASC LIMIT 20 OFFSET 0;

SELECT id, email FROM users WHERE email LIKE '%@example.com';

-- Join
SELECT u.name, p.title
FROM users u
JOIN posts p ON p.author_id = u.id
WHERE u.id = 1;

-- Aggregate
SELECT author_id, COUNT(*) AS post_count
FROM posts
GROUP BY author_id
HAVING COUNT(*) > 5;

// node-postgres
const { rows } = await pool.query(
  "SELECT * FROM users WHERE age >= $1 ORDER BY name LIMIT $2",
  [18, 20]
);
```

3.4 Update

```
UPDATE users SET name = 'Jane Doe' WHERE id = 1;

UPDATE users SET status = 'inactive' WHERE last_login < NOW() - INTERVAL '90 days';

-- Update and return the changed rows
UPDATE users SET name = 'Jane Doe' WHERE id = 1 RETURNING *;

-- Upsert
INSERT INTO users (email, name) VALUES ('jane@example.com', 'Jane')
ON CONFLICT (email) DO UPDATE SET name = EXCLUDED.name;

// node-postgres
await pool.query("UPDATE users SET name = $1 WHERE id = $2", ["Jane Doe", 1]);
```

3.5 Delete

```
DELETE FROM users WHERE id = 1;

DELETE FROM users WHERE status = 'inactive';

-- Delete and return the removed rows
DELETE FROM users WHERE id = 1 RETURNING *;

TRUNCATE TABLE sessions; -- fast wipe of all rows

// node-postgres
await pool.query("DELETE FROM users WHERE id = $1", [1]);
```

3.6 Transactions

```
BEGIN;
UPDATE accounts SET balance = balance - 100 WHERE id = 1;
UPDATE accounts SET balance = balance + 100 WHERE id = 2;
COMMIT; -- or ROLLBACK on error

// node-postgres
const client = await pool.connect();
try {
  await client.query("BEGIN");
  await client.query("UPDATE accounts SET balance = balance - 100 WHERE id = $1", [1]);
  await client.query("UPDATE accounts SET balance = balance + 100 WHERE id = $1", [2]);
  await client.query("COMMIT");
} catch (e) {
  await client.query("ROLLBACK");
  throw e;
} finally {
  client.release();
}
```

3.7 psql Meta-commands

Command	Purpose
\l	List databases
\c dbname	Connect to a database
\dt	List tables

\d tablename	Describe a table's columns/indexes
\du	List roles/users
\q	Quit psql

4 | SQL Core Reference

4.1 Data Definition (DDL)

```
CREATE TABLE users (  
  id          SERIAL PRIMARY KEY,  
  email       VARCHAR(255) UNIQUE NOT NULL,  
  name        VARCHAR(255),  
  created_at  TIMESTAMP DEFAULT NOW()  
);  
  
ALTER TABLE users ADD COLUMN age INT;  
ALTER TABLE users DROP COLUMN age;  
ALTER TABLE users RENAME COLUMN name TO full_name;  
ALTER TABLE users ALTER COLUMN email SET NOT NULL;  
  
CREATE INDEX idx_users_email ON users (email);  
DROP INDEX idx_users_email;  
  
DROP TABLE users;
```

4.2 Data Manipulation (DML)

```
INSERT INTO users (email, name) VALUES ('a@x.com', 'A');  
SELECT * FROM users WHERE id = 1;  
UPDATE users SET name = 'A2' WHERE id = 1;  
DELETE FROM users WHERE id = 1;
```

4.3 Filtering & Sorting

```
SELECT * FROM users  
WHERE age BETWEEN 18 AND 65  
      AND (status = 'active' OR status IS NULL)  
      AND email LIKE '%@example.com'  
ORDER BY created_at DESC  
LIMIT 10 OFFSET 20;
```

4.4 Joins

```
-- INNER JOIN: only rows matching in both tables  
SELECT * FROM orders o INNER JOIN users u ON o.user_id = u.id;  
  
-- LEFT JOIN: all rows from the left table, matched or not  
SELECT * FROM users u LEFT JOIN orders o ON o.user_id = u.id;  
  
-- RIGHT JOIN: all rows from the right table, matched or not  
SELECT * FROM orders o RIGHT JOIN users u ON o.user_id = u.id;  
  
-- FULL OUTER JOIN: all rows from both tables  
SELECT * FROM users u FULL OUTER JOIN orders o ON o.user_id = u.id;
```

4.5 Aggregates & Grouping

```
SELECT COUNT(*), AVG(price), MIN(price), MAX(price), SUM(price)  
FROM products;
```

```
SELECT category, COUNT(*) AS n
FROM products
GROUP BY category
HAVING COUNT(*) > 3
ORDER BY n DESC;
```

4.6 Subqueries & CTEs

```
-- Subquery
SELECT * FROM users
WHERE id IN (SELECT user_id FROM orders WHERE total > 100);

-- Common table expression
WITH big_orders AS (
  SELECT user_id, SUM(total) AS spend
  FROM orders
  GROUP BY user_id
)
SELECT u.name, b.spend
FROM users u
JOIN big_orders b ON b.user_id = u.id
WHERE b.spend > 500;
```

4.7 Constraints

Constraint	Purpose
PRIMARY KEY	Uniquely identifies each row
FOREIGN KEY	Enforces a relationship to another table
UNIQUE	No duplicate values in the column
NOT NULL	Column cannot be empty
CHECK (expr)	Row must satisfy a boolean expression
DEFAULT value	Value used when none is supplied

Prepared as a quick command reference for Prisma, MongoDB, PostgreSQL and core SQL. Verify syntax against the current docs for your exact driver/database version before using in production.